

Problem and Motivation

GPT-2 is a decoder-only transformer language model that processes sequences of tokens through a stack of identical layers, each comprising a multi-head self-attention sublayer followed by a position-wise feed-forward network (MLP). I propose to implement an accelerator for the training process of the GPT-2 LLM. The training algorithm includes both forward and backwards passes, which increases the number of matrix multiplications required per step from one to three.

Profiling the algorithm determined that the three multiplication operations combine to represent 86% of computation time for the training algorithm. My accelerator therefore primarily targets matrix multiplication. Targeting exclusively matrix multiplication results in a theoretical speedup of 7.14x limit with infinite speedup of the targeted kernel. Targeting specifically matrix multiplication allows for construction of a much smaller accelerator than would be necessary to speed up the entire design, as it allows data to be streamed in and out, rather than having to hold entire matrices on chip in SRAM if portions of the algorithm like normalization were included as acceleration targets.

The three types of matrix multiplication exhibit identical arithmetic structure. They differ only in which operand is held stationary and which is streamed, which makes a weight-stationary systolic array a natural architectural fit. A set of weights can be loaded, and 256 activation rows streamed through it, allowing excellent natural tiling of the multiplication. The large size of the matrices ($[256 \times 768] \times [768 \times 3072]$ for forward matrix multiplication) result in high arithmetic intensity, which in turn implies a high theoretical throughput speedup without increasing the memory bandwidth.

Roofline Analysis

The baseline system tested against is my laptop, which has a maximum compute throughput of 204.8 GFLOPS and a maximum memory bandwidth of 38.4 GB/s, for a ridge point of 5.33 FLOP/B. This places the matrix multiplication portion of the original algorithm, with an arithmetic intensity of 33.5 FLOP/B, deeply in the compute-bound portion of the graph (Figure 1). The primary goal of my accelerator was therefore to provide sufficient compute bandwidth to exploit that high arithmetic intensity, shifting the bottleneck from compute capacity to memory bandwidth, where throughput scales directly with the AXI bus and is ultimately limited by DRAM bandwidth.

With a 32×32 systolic array as its compute core, the maximum throughput of my accelerator was determined to be 1.24 TFLOPS at 606 MHz, while the data bandwidth of the 512-bit AXI-Stream bus was determined to be 38.75 GB/s. This places the ridge point of my accelerator at 32.3 FLOP/B, much closer to the arithmetic intensity of the kernel.

Interestingly, the matrix multiplication implementation on the accelerator has slightly worse arithmetic intensity than the original CPU implementation. This is due to the design decision to keep on-chip SRAM buffers as small as possible, which results in better cache behavior on the CPU than on the accelerator. This in turn causes more data re-streaming than is present in the CPU implementation, lowering the arithmetic intensity from 33.5 FLOP/B in the original version to 25.8 FLOP/B on the accelerator. This moves the implementation into the memory-bound portion of the roofline, though at a much higher absolute throughput than the original.

The final effective throughput of my design is 563 GFLOP/s, which represents a 19.4× speedup on the matrix multiplication portion compared to the original CPU implementation. This places the accelerated kernel below the memory-bound ceiling of the roofline, which has a theoretical maximum of approximately 1.00 TFLOPS at this arithmetic intensity ($25.8 \text{ FLOP/B} \times 38.75 \text{ GB/s}$). Reaching this peak would require full AXI bus utilization on every clock cycle, which is not the case in the current design: the AXI bus sits idle while accumulated results are transferred back to the host between tiles.

Reaching the memory-bound roofline ceiling would therefore require double-buffering the accumulator SRAM, so that new tiles could be computed while the previous tile's results are transferred out. This would require doubling the accumulator SRAM from its current 32 KB, already the largest on-chip buffer in the design. I instead opted to prioritize power and area over peak throughput, because at 19.4× matrix multiplication speedup, further improvements to matrix multiplication throughput yield diminishing system-level returns due to Amdahl's Law. Matrix multiplication has already been reduced from approximately 86% of the original runtime to 24% of the accelerated projected runtime, and is therefore no longer the dominant bottleneck.

Dataflow and Architecture

My design follows the traditional design of compute intensive kernel hardware accelerator, consisting of an input bus, which will be discussed in a future section, an SRAM buffer for input values, a compute core, output SRAM, and an interface to return data to the host. My design includes one component not in the traditional design, namely a 9 stage pipeline that implements GELU operations on the returning data before sending it back to the host. A block diagram of my design is figure 2.

The interface consists of a 512 bit AXI stream bus, which streams into one of two buffers. The weight buffer is 4KB SRAM split into two banks. Each bank is 2KB, enough space to hold the weights to completely fill the systolic array. The two banks allow double buffering, where weight data can be streamed into one buffer while the contents of the other buffer are loaded into the systolic array, thereby avoiding sequential weight streaming and loading. The other buffer is the 2KB activation buffer, which holds activations to be fed into the systolic array. It is necessary because activations must be fed into the array in a staggered fashion, which requires the first activation column to be present in memory for the full 32 cycles of its entry into the systolic array. By the time the last element of this column is loaded, the first element of the 32nd column is also loaded, which dictates the required size of the buffer. The 32 columns of 32, 2 byte elements total to 2KB, the size of the activation buffer.

The compute core of my design is a 32x32 weight stationary systolic array. The decision to use a weight stationary design is based on the limited data bandwidth of the design. The 512 bit AXI bus used to transfer data to the accelerator has a ~.5% higher bandwidth than the DRAM that backs it. This means that there is no way to lift the accelerator's current bandwidth limit. Shifting from a weight stationary to output stationary design would require streaming both weights and activations, which would double the required bandwidth of the system. Existing bandwidth is already fully utilized to stream activations to the systolic array, so an output stationary design would require dropping from a 32x32 systolic array down to a 16x16 implementation, a 4x throughput reduction. The weight stationary model is not without its downsides, however. Weight loading costs 32 cycles whenever the weights must be swapped out, a cost which would drop to zero with an output stationary implementation.

There are several other factors which reduce the efficiency of the systolic array, which in practice only reaches 60% effective utilization in my design. The first is mentioned above, that during weight loading no computations are being performed. This adds 32 cycles to every weight tile computation.

Additional per weight tile losses occur due to the wavefront effect, 66 unavoidable fill/drain cycles that occur for every tile as the first data moves through the systolic array and the last data moves out of it. During this time the array is only partially occupied, which reduces its efficiency. The net effect of these partial usage cycles is 34 full cycles lost. Finally, a number of cycles are lost due to the systolic array being inactive during write back. Unlike the weight SRAM, the accumulator SRAM is not double buffered. This means that additional data cannot be written to it while it is being transmitted back to the host. This results in 512 lost cycles per full tile while transmission occurs. This cost is only paid once every full tile computation, which consists of 24 weight tiles, so the actual breakdown in number of cycles lost per full tile is:

	Cycles per Tile	% of Total
Weight Loading	768	36.6
Stagger Overhead	816	38.9
Readback	512	24.4
Total Overhead	2,096	100

As previously mentioned, outputs from the systolic array are stored in the accumulator SRAM for reuse as inputs to the systolic array and for eventual return to the host device. The accumulator SRAM is not double buffered due to its size. It is 32KB and dominates the SRAM capacity of the chip. The decision was made to optimize in this case for reducing the size and power consumption of the chip, rather than the pure throughput, as the readback overhead is only 24% of the total lost cycles on the systolic array and double buffering would have doubled the size of the SRAM.

This design targets three different matrix multiplication operations, the forward, weight gradient, and input gradient operations. Immediately following the forward multiplication in the algorithm is GELU computation for each of the outputs. Given that the data is already present, I decided to implement the GELU computation in hardware, as it requires no additional data movement, and, since it is pipelined, results in near infinite effective speedup. In software GELU is computed:

$$GELU(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} \cdot (x + 0.044715 \cdot x^3) \right) \right)$$

This would not be feasible in hardware, as the tanh function would require a massive lookup table to compute in an acceptable amount of time. I opted to approximate this equation with the following Padé [3/2] rational approximation.

$$\tanh(u) \approx u \cdot \frac{1485 + 171 \cdot u^2 + 2 \cdot u^4}{1485 + 666 \cdot u^2 + 26 \cdot u^4}$$

This is implemented as a 9 stage pipeline, broken down as follows:

Stage	Operation	Max Multiplier Depth
Input → R0	Compute x^2 and x^3	2 sequential BF16 muls
R0 → R0b	Compute $0.044715 \cdot x^3$, inner sum, and $\sqrt{(2/\pi)} \cdot \text{sum}$	2 sequential BF16 muls
R0b → R1	Evaluate u^2 operator	1 BF16 mul
R1 → R1b	Compute u^4 , $171 \cdot u^2$, and $666 \cdot u^2$ in parallel	1 parallel multiplier level
R1b → R2	Evaluate numerator and denominator polynomials	2 sequential BF16 muls
R2 → R3	Newton-Raphson Iteration 1 (Reciprocal seed + initial refinement)	2 sequential BF16 muls
R3 → R3b	Newton-Raphson Iteration 2 (Secondary refinement)	2 sequential BF16 muls
R3b → Out	Compute $\tanh \cdot \text{num}$, $0.5 \cdot x$, saturation muxing, and final product	2 sequential BF16 muls

The final destination for data is the AXI stream output bus, where it is returned to the host for further processing. It should be noted that the aforementioned GELU pipeline is only used for the forward matrix multiplication, and that for the weight gradient and input gradient multiplications it is bypassed and the results of matrix multiplication are sent directly to the output bus.

Hardware Interface

The accelerator uses a 512-bit AXI-Stream bus. At 606 MHz, it provides 38.7 GB/s of bandwidth, effectively matching DRAM throughput. AXI-Stream was chosen over PCIe for its design simplicity at this bandwidth scale, while lower-throughput protocols (such as SPI or I2C) were fundamentally insufficient.

The decision to maximize bandwidth is based on the roofline analysis from the previous section, which indicated that the accelerated kernel implementation is memory bound rather than compute bound as it was in the CPU implementation. This is due to several factors. First is that compute acceleration lifts the throughput maximum by $\sim 6x$, which shifts the ridgepoint to the right by the same amount given the same memory bandwidth. Second, is that the accelerator algorithm is slightly less efficient in terms of data reuse than the original design, which shifts its arithmetic intensity to the left. The net result is that optimizing throughput requires maximizing the bandwidth of the interface and ensuring that it is always active.

The activity ratio of the interface is 87%, which is also the ratio of effective throughput to potential throughput from my roofline analysis. There are two reasons that the bus is not fully utilized. First is that computation is paused while data is being returned to the host. During this time there is nothing to stream, so the receiver bus is idle as well. The reason for this is that the output SRAM is not double buffered to save space and power on the chip, which prevents reading to and writing from it simultaneously. If double buffering were implemented, this source of utilization loss would be eliminated. It represents $\sim 38\%$ of lost effective bandwidth. The remaining 62% are unavoidable and can be attributed to the drain time of the systolic array. At the end of every weight tile there are 32 cycles in which computation is still occurring, but all tile data has already been sent. This can be reduced by increasing batch size, but that is independent of hardware design.

Precision and Data Format

The original CPU implementation of my algorithm uses FP32 for all floating point operations. In order to reduce the pressure on the AXI bus I opted to use BF16 floating point for data transfers to the accelerator, as well as for all multiplication within the design. Accumulation is still performed using FP32. The decision to use this data format was based on comparisons of output weights after a single training step to the outputs from the original CPU design, with the goal of keeping the mean relative distance between the two at less than 1%. At the time of testing, only the forward pass matrix multiplication was implemented, so a more stringent requirement of less than .3% mean relative distance was required, with the assumption that the backwards pass would result in at least as much deviation as the forward pass. The results of that analysis are as follows:

Configuration	Mean Rel. Δ	Cosine Similarity
BF16 $\times$ FP32 (implemented)	0.0183%	0.99999999
INT16 $\times$ FP32	0.0173%	0.99999999
FP8 $\times$ FP32	0.5733%	0.99999966
INT8 $\times$ FP32	0.5710%	0.99999966
BF16 $\times$ BF16 (no FP32 accum)	0.8135%	0.9999995
Final hardware	0.8011%	0.99999951

The best performing configuration for the forward pass was INT16 $\times$ FP32, but this option was discarded as its strong performance was only due to low dynamic range of the values in the forward pass. This is not the case for the backward pass multiplications, where it was expected to produce a sharp increase in mean relative distance. FP8 and INT8 were both disqualified as they produced a mean relative difference of greater than .5%, which would ensure that the net mean relative difference was greater than 1% once backwards pass multiplication was implemented. Using BF16 accumulation was also rejected for the same reason.

The GELU pipeline also uses BF16 multiplication and FP32 addition. The use of BF16 multiplication and FP32 accumulation allows for the GELU pipeline to be implemented in 9 pipeline stages. If FP32 multiplication were used, it would require further pipelining to isolate each multiplication operation into its own clock cycle. This would improve precision, as in the current design the BF16 operations effectively truncate the outputs of the systolic array multiplication. Since the current implementation still results in a mean relative difference of less than 1% per training step, it was accepted even though it results in precision loss.

The final hardware measurement was included in the table, but should not be compared to the other values, as the conditions under which the data was collected was different than the previous table rows. These data were collected after all matrix multiplication operations and the forward pass GELU had been moved to the accelerator, and indicates that the final implementation achieves the original goal of keeping the mean relative difference between the output weights of a single pass of the accelerator and a single pass of the original software implementation less than 1%.

Verification

There was a three stage verification process that was implemented for this design. Each module was tested upon completion to determine if it was functioning correctly. A second round of testing was completed at the top level of the design, to determine if the interconnections were functioning properly, and to test the top level implementation. Finally, the functional behavior of the hardware was tested by integrating the simulated design directly into the software training loop.

Individual Verilator testbenches were written for each submodule of the design. These included the processing element (PE), the systolic array, the weight and activation SRAMs, the AXI input interface, the AXI readback interface, and the controller. Each test was constructed to isolate the behavior of a single module by driving its inputs directly and checking its outputs against analytically computed expected values. This approach allowed functional errors to be identified and corrected close to the point of introduction, before they could propagate into harder-to-diagnose failures at higher levels of integration.

Following module-level verification, a comprehensive top-level testbench was developed targeting the complete top module. The testbench was compiled and simulated using Verilator. Sixteen tests were executed, progressing from simple functional checks to boundary, stress, and timing-sensitive regression cases:

Test	Description	Purpose
1	$M=4, W=1, \text{act}[m]=m+1 \rightarrow \text{out}[m]=\text{ROWS} \times (m+1)$	Basic streaming correctness; verifies per-row scaling
2	Two K-tiles, $W=1, \text{act}=1 \rightarrow \text{out}=2 \times \text{ROWS}$	Multi-tile accumulation via ping-pong weight banks
3	$M=16, W=2, \text{act}=1 \rightarrow \text{out}=2 \times \text{ROWS}$	Larger M count; verifies stagger at deeper pipeline fill
4	Backward mode ($\text{mode}=1$), $W=1, \text{act}=1 \rightarrow \text{out}=\text{ROWS}$	Backward datapath
5	$\text{Bias}=0.5, W=1, \text{act}=1 \rightarrow \text{out}=\text{ROWS}+0.5$	Uniform bias addition via bias SRAM
6	GELU only, no bias $\rightarrow \text{GELU}(\text{ROWS})$	GELU post-processing on positive input
7	$\text{Bias}=0.5 + \text{GELU} \rightarrow \text{GELU}(\text{ROWS}+0.5)$	Combined bias and GELU path
8	Identity weight matrix $\rightarrow \text{out}[m][c]=c+1$	Regression for weight-row reversal fix
9	$M=1$ (minimum)	Lower boundary of M_count; single-row drain
10	$M=64$ (stress)	Verifies accumulator address sequencing at scale
11	$\text{act}=-1, W=1 \rightarrow \text{out}=-\text{ROWS}$	Negative activation sign handling in BF16
12	$\text{bias}[c]=c+1 \rightarrow \text{out}[m][c]=\text{ROWS}+c+1$	Per-column bias; each column gets its own bias value
13	$W=-1/\text{ROWS}, \text{act}=1 \rightarrow \text{raw}=-1, \text{GELU}(-1) \approx -0.159$	GELU correctness on a negative pre-activation value
14	$M=256$ (M_MAX)	Full accumulator depth; address counter rollover
15	4 K-tile accumulation $\rightarrow \text{out}=4 \times \text{ROWS}$	Accumulation correctness beyond two tiles

16	Ping-pong preloading: W=1 bank 0, preload W=2 bank 1 → out=3×ROWS	Preloading timing; AXI write to idle bank during active computation
----	---	---

The final and most stringent verification stage embedded the cycle-accurate Verilator model directly into the GPT-2 C training source code. Verilator compiled the sys_top RTL into a C++ class (Vsys_top), which was instantiated in a hardware abstraction layer (accel_hal_systolic_vrl.cpp). The HAL drives the DUT's AXI-Stream slave port, control signals, and readback interface on a cycle-by-cycle basis using the same tick() primitive used in the standalone testbenches, but now called from within the training loop rather than from a dedicated testbench. From the perspective of the training code, the Verilator model is indistinguishable from a real hardware device: it receives weight tiles and activation slices through the same HAL API and returns FP32 results through the same readback interface.

Gradient outputs were compared against the original CPU-only reference implementation. While these values were expected to have some deviation from the originals due to precision reduction in the design, significant (>1%) differences in mean relative distance were viewed as erroneous and a sign that there was an error in the implementation.

Synthesis Results

Synthesis was performed using the Cadence tool chain, specifically Genus 23.10, targeting the ASAP7 7nm RTV standard cells. The rationale behind using ASAP7 rather than SKY130, which is more usual for a tapeout, is that I wanted the timing, power, and size values determined in synthesis to better reflect the modern accelerator and GPU space, where cells may be as small as 2nm for cutting edge fabrication. Using 7nm cells puts me in the same order of magnitude as real designs, whereas the results of SKY130 synthesis could be as many as two orders of magnitude off just due to cell size.

Total estimated power is 0.900 W. Sequential logic (registers and flip-flops) accounts for 0.367 W (40.8%), while combinational logic accounts for 0.533 W (59.2%). The total cell area is 385,240.837 μm^2 , distributed across three primary structures. The accumulator SRAM accounts for 144,918.464 μm^2 (37.6% of total). The systolic array occupies 171,343.154 μm^2 (44.5%), with each of the 1,024 PEs contributing 163.763 μm^2 . The remaining 17.9% covers the AXI-Stream input controller, readback unit (including 32 GELU pipeline instances), weight SRAMs, and interconnect.

Of note is the high contribution of sequential logic to power and accumulator SRAM to the area. The reason for both is a limitation of ASAP7. ASAP7 does not have SRAM cells, and as such I was forced to synthesize all on chip storage as flip-flops instead. Flip flops consume 10-15x as much power as SRAM and 5-10x as much space. While I do not have a direct breakdown of how much of the sequential logic is required to be implemented as flip flops, the vast majority could be replaced with SRAM cells if they were available, which would conservatively reduce the sequential logic power consumption by a factor of 10 down to .0367 W, and the size of the accumulator SRAM by a factor of 5 down to 28983.69 μm^2 . This would bring the total power consumption down to 0.570 W, a reduction of 37%, and area down to 269,306.063 μm^2 , for a reduction of 30%.

The synthesis clock period target was 1650ps, for a frequency of ~606 MHz. Prior to the inclusion of the fused GELU pipeline, synthesis resulted in timing closure at this frequency, but after its inclusion, timing failed with a worst negative slack of -.9ps and a total negative slack of -85.3 ps, with 154 violating paths. All failing paths are in the GELU pipeline. The critical path consists of a BF16 multiply unit followed by an FP32 adder, both of the type used in the MAC units in the systolic array.

Additional pipeline stages in the GELU pipeline would solve these problems, as it would be simple to place registers between each of the compute units. Based on how limited the timing failure was, and because the critical path is effectively the same as is present in the systolic array for each MAC unit, this is unnecessary. One additional round of route optimization would close these timing failures, and if this failed, relaxing the clock period constraint by one picosecond would have a negligible impact on throughput of the design. Since no additional design modifications are necessary to achieve timing closure, and the timing failure is so small compared to the clock period ($\sim 0.06\%$), I am treating these results as effective closure with a 1651ps clock period, which has virtually no impact on the final performance results compared to the original 1650ps.

Benchmark Results

All benchmarks were run on a single complete forward and backward training pass over GPT-2 small (12 transformer layers, $C=768$, 12 attention heads) with batch size $B=4$ and sequence length $T=64$. CPU timing uses the original software implementation with a warmup step, as typically the first step performs significantly worse than subsequent steps. Energy consumption information is acquired via the Linux RAPL sysfs interface, with readings taken immediately before and after each matmul call and accumulated across the full training step.

Accelerator timing is derived from a 64-bit cycle counter in the Verilator simulation that increments every simulated clock edge. The hardware abstraction layer (HAL) accumulates forward and backward cycle totals separately. Compute time is obtained by dividing the total simulated cycle count by the synthesized clock frequency of 606 MHz. Though the entire accelerated training layer is executed, execution time for the non-accelerated CPU work is taken directly from the software backend run and added to the projected hardware acceleration time. This is done because Verilator execution evicts cache data used by the CPU, which results in a much slower execution time for the CPU portions of the accelerated version, which is a function of the simulation method, rather than an actual behavior of the accelerated version of the algorithm.

The results of benchmark testing are as follows:

Metric	Software	Accelerated	Ratio
Matrix multiplication time	4.5055 s	0.2317 s	19.4×
Non-matmul CPU time	0.7376 s	0.7376 s	1.0×
Total time	5.2431 s	0.9694 s	5.41×

The speedup for matrix multiplication is 19.4x, resulting in a total speedup of 5.41x. Matrix multiplication is reduced from 86% of execution time in the software implementation to 23.9% in the accelerated version. This implies significant diminishing returns to further matrix multiplication speedup in terms of improvements in total training time. A perfect accelerator with infinite speedup would only improve the full training speedup from 5.41x to 7.14x. While there is still room for improvement on this front, it may be more efficient to find methods to accelerate other portions of the algorithm, where a smaller improvement would have a greater overall impact.

As mentioned in the roofline analysis section, this implementation is currently memory bound, and is not fully utilizing the full memory bandwidth of the AXI bus. This reduces the accelerator

throughput to 563 GFLOP/s from a memory bound limit of ~ 1 TFLOP/s and a compute bound limit of 1.24 TFLOP/s. It should be noted that neither of these limits are truly attainable. In the case of the compute limit, fill drain delays cannot be avoided, and can only be amortized by increasing the tile size. Weight loading time also impacts compute utilization, as the systolic array is idle while new weights are being loaded into it. This can only be avoided by moving to an output stationary implementation, which would double the bandwidth requirements of the chip, or mitigated by increasing the batch size. There is one factor in the utilization loss that is avoidable, namely dead time on write back. Currently, no computation is performed during the writeback phase as the accumulator SRAM is not double buffered. Double buffering would significantly increase the size of the accelerator, but would theoretically raise the throughput by $\sim 6\%$, resulting in a matrix multiplication speedup of 20.5x and a corresponding full algorithm speedup of 5.48x.

Energy consumption comparisons between the CPU and accelerator implementations are based on a combination of sources. Accelerator power consumption is based on execution time and power consumption information from synthesis. CPU energy consumption was determined via Intel RAPL, measuring exclusively matrix multiplication portions of the algorithm. RAPL measurements summed to 90.04J for the full training step on the CPU. For the accelerator, the total energy consumption was determined by multiplying the measured execution time for all three matrix operations, .2317s, by the power consumption of the accelerator chip, .9W, for a total energy consumption value of .2086J. This represents a 432x energy consumption reduction between the CPU and accelerator versions of the kernel. It should be noted that neither energy consumption measurement considered DRAM or AXI transfer costs, which would be roughly equivalent for each, but would significantly reduce the total energy consumption reduction value. In fact, the structure of the accelerator means that accelerator DRAM traffic is slightly higher than CPU DRAM traffic, which would further erode this advantage.

Issues

The primary architectural constraint stemmed from the ASAP7 library's lack of native memory primitives. Synthesizing all storage from flip-flops inflated area and power, preventing performance optimizations like a double-buffered accumulator or an activation cache. A commercial 7nm FinFET PDK with native SRAM compilers would eliminate this bottleneck.

Another improvement that I would make is to modify the state machine in my design to allow loading of weights during the draining process of the previous tile. This would allow overlapping the dead compute cycles of the two processes, and keep the bus busy during the time it would usually be idle during the drain. This change would eliminate 32 dead cycles per weight tile and therefore improve performance by $\sim 9.6\%$.

My implementation has gone through a number of iterations, reflecting multiple design paths that ultimately proved ineffective or not feasible. My final design actually matches my initial design in the broad sense, they are both 32x32 systolic arrays, but my original design was built before I had a full understanding of systolic array processing, and thus was profoundly inefficient. I therefore opted to implement a vectored MAC implementation, first with 128 MACs and then with 512 when the projected speedup for the 128 MAC version proved anemic. The 512 vectored implementation ran into problems when I realized that the forward matrix multiplication dimensions did not divide evenly into 512, which meant that the accelerator would not be fully utilized for part of the multiplication. This led me back to a systolic design, where I implemented a 32x16, 16x16, and 32x32 implementation, eventually deciding on the 32x32 implementation to maximize throughput.

Additional problems occurred during the design, including many problems building a `genus.tcl` file that would function correctly with the 23.10 version of the software. The reason for transitioning to Genus was due to Openlane 2 not supporting ASAP7 natively, and the computer I was working with not having enough RAM to synthesize my design. This led me to migrate to a PSU lab server (Centos7) and to the Cadence tool chain.

Figures

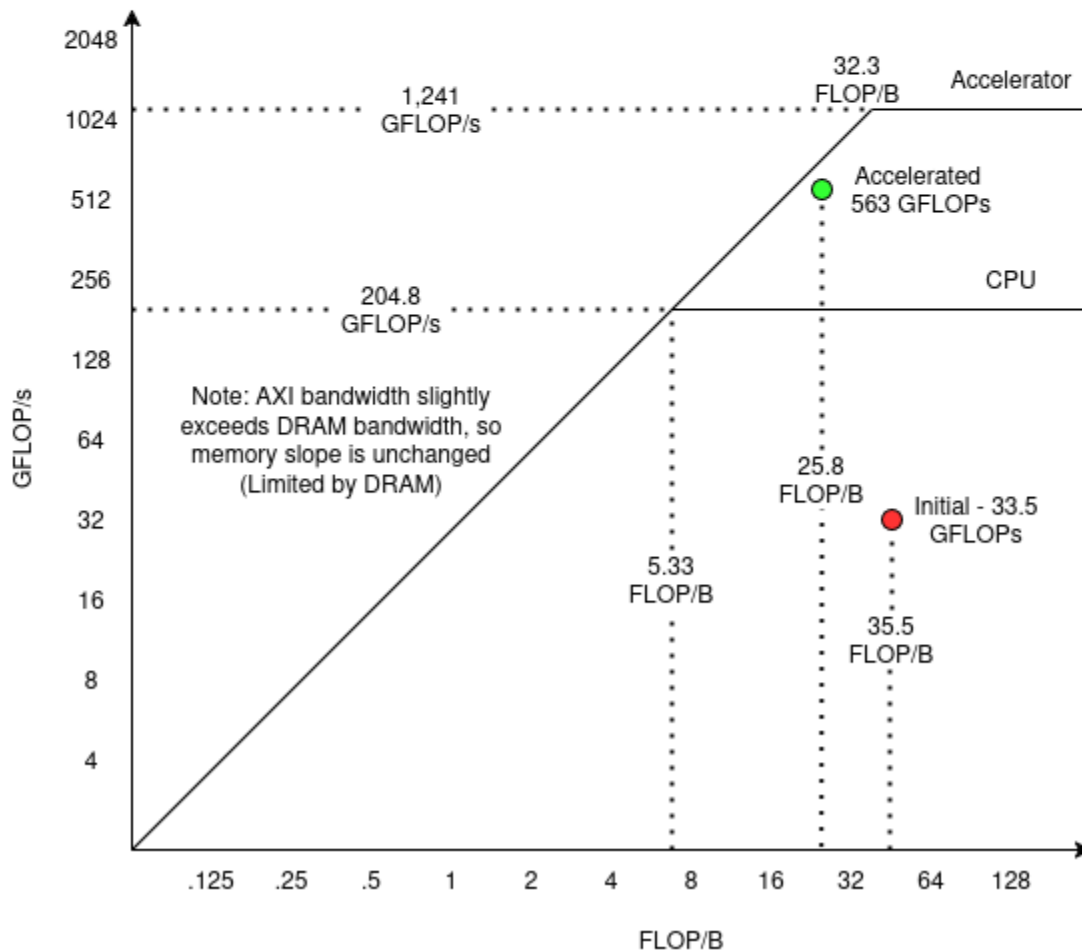


Figure 1.

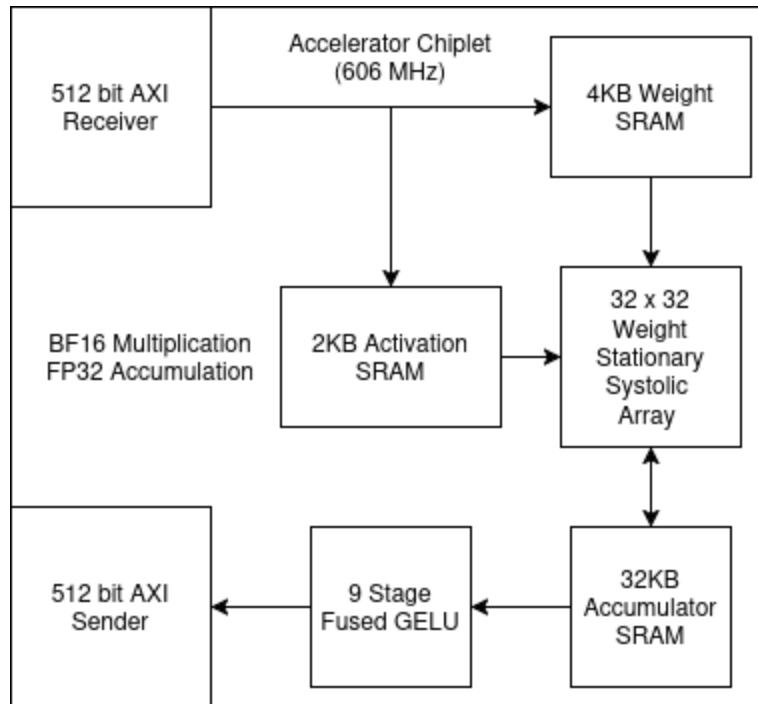


Figure 2.

Acknowledgments

This project is based on a C implementation of GPT2 training by Andrej Karpathy.
<https://github.com/karpathy/llm.c>